

Lecture 1: Greedy Algorithms

Introduction to Algorithms (CS 4820)

Topic: The Greedy Algorithm Design Paradigm

1. What Is a Greedy Algorithm?

A greedy algorithm builds a solution piece by piece, always choosing the option that looks best *right now* — the locally optimal choice — in the hope that this leads to a globally optimal solution. It never reconsiders or backtracks on choices it has already made.

The defining characteristic: at each step, commit to the choice that seems best at that moment, without worrying about future consequences.

Greedy is not always correct. For some problems it produces an optimal answer; for others a reasonable but suboptimal one. The hard part is *proving* a greedy approach actually yields the optimum.

2. When Does Greedy Work? Two Key Properties

A problem is a good candidate for a greedy solution when it has both:

- **Greedy Choice Property.** A globally optimal solution can be reached by making a locally optimal (greedy) choice at each step. The locally best choice is part of *some* optimal solution — you never have to undo it.
- **Optimal Substructure.** An optimal solution contains within it optimal solutions to subproblems. After the greedy choice, what remains is a smaller instance of the same problem.

If both hold, greedy finds the optimum. Optimal substructure is shared with dynamic programming; the difference is that greedy commits to one choice without first solving all subproblems.

3. Canonical Example: Activity Selection (Interval Scheduling)

Problem. Given n activities, each with a start time s_i and finish time f_i , and only one activity running at a time, select the maximum number of mutually compatible (non-overlapping) activities.

Greedy strategy. Sort activities by *finish time*, earliest first. Repeatedly pick the next activity whose start time is $\geq$ the finish time of the last one selected.

Why finish time? Choosing the activity that finishes earliest leaves the maximum remaining time for other activities — it greedily preserves the most future opportunity.

```
Sort activities by finish time
selected = [first activity]
last_finish = f_1
for each remaining activity i in order:
    if s_i >= last_finish:
        add activity i to selected
        last_finish = f_i
return selected
```

Running time. $O(n \log n)$, dominated by the sort; the selection scan is $O(n)$.

Common wrong greedy choices (these do NOT give the optimum): shortest activity first, earliest start time, or fewest conflicts. Only “earliest finish time” is provably optimal here.

4. Proving Greedy Correctness: The Exchange Argument

The standard technique for proving a greedy algorithm optimal is the **exchange argument** (related to “greedy stays ahead”).

The idea: assume an optimal solution that differs from the greedy one. Show you can swap (exchange) an element of the optimal solution for the greedy choice *without making it worse*. Repeating this transforms the optimal solution into the greedy one while preserving optimality — proving the greedy solution is also optimal.

Greedy stays ahead variant: prove by induction that after each step, the greedy solution is at least as far along (by some measure) as any other solution.

5. Huffman Coding

Problem. Given characters with frequencies, build a binary prefix code (no codeword is a prefix of another) minimizing total encoded length.

Greedy strategy. Repeatedly take the two lowest-frequency nodes, merge them into a new node whose frequency is their sum, and reinsert. Continue until one tree remains. Low-frequency characters end up deeper (longer codes); high-frequency characters end up shallower (shorter codes).

- **Data structure:** a min-priority queue (binary heap) to extract the two minimums.
- **Running time:** $O(n \log n)$ for n characters with a binary heap.

Huffman coding is provably optimal among prefix codes — a classic exchange-argument proof.

6. Other Notable Greedy Algorithms

- **Dijkstra’s shortest path** — greedily selects the unvisited vertex with smallest known distance. Correct for non-negative edge weights only.
- **Kruskal’s MST** — greedily adds the cheapest edge that does not form a cycle.
- **Prim’s MST** — greedily grows a tree by adding the cheapest edge leaving it.
- **Fractional Knapsack** — greedily take items by highest value-to-weight ratio. Optimal for the *fractional* version but NOT for 0/1 knapsack.

7. Key Warning: Greedy Fails on 0/1 Knapsack

For 0/1 knapsack (each item taken whole or not at all), the greedy value-to-weight ratio strategy can fail: a high-ratio small item can crowd out a better combination of others. 0/1 knapsack requires dynamic programming for a guaranteed optimum. This is the canonical “greedy is tempting but wrong” example and a frequent exam question.

8. Summary

Greedy algorithms are fast and simple, often $O(n \log n)$, and elegant when they work. The challenge is never the implementation — it's recognizing whether the greedy choice property and optimal substructure hold, and proving it (usually via an exchange argument). When in doubt, greedy is a hypothesis to be proven, not assumed.

Exam reminder: Be ready to (a) state the two properties required for greedy correctness, (b) trace activity selection by finish time, and (c) explain why greedy fails on 0/1 knapsack but works on fractional knapsack.